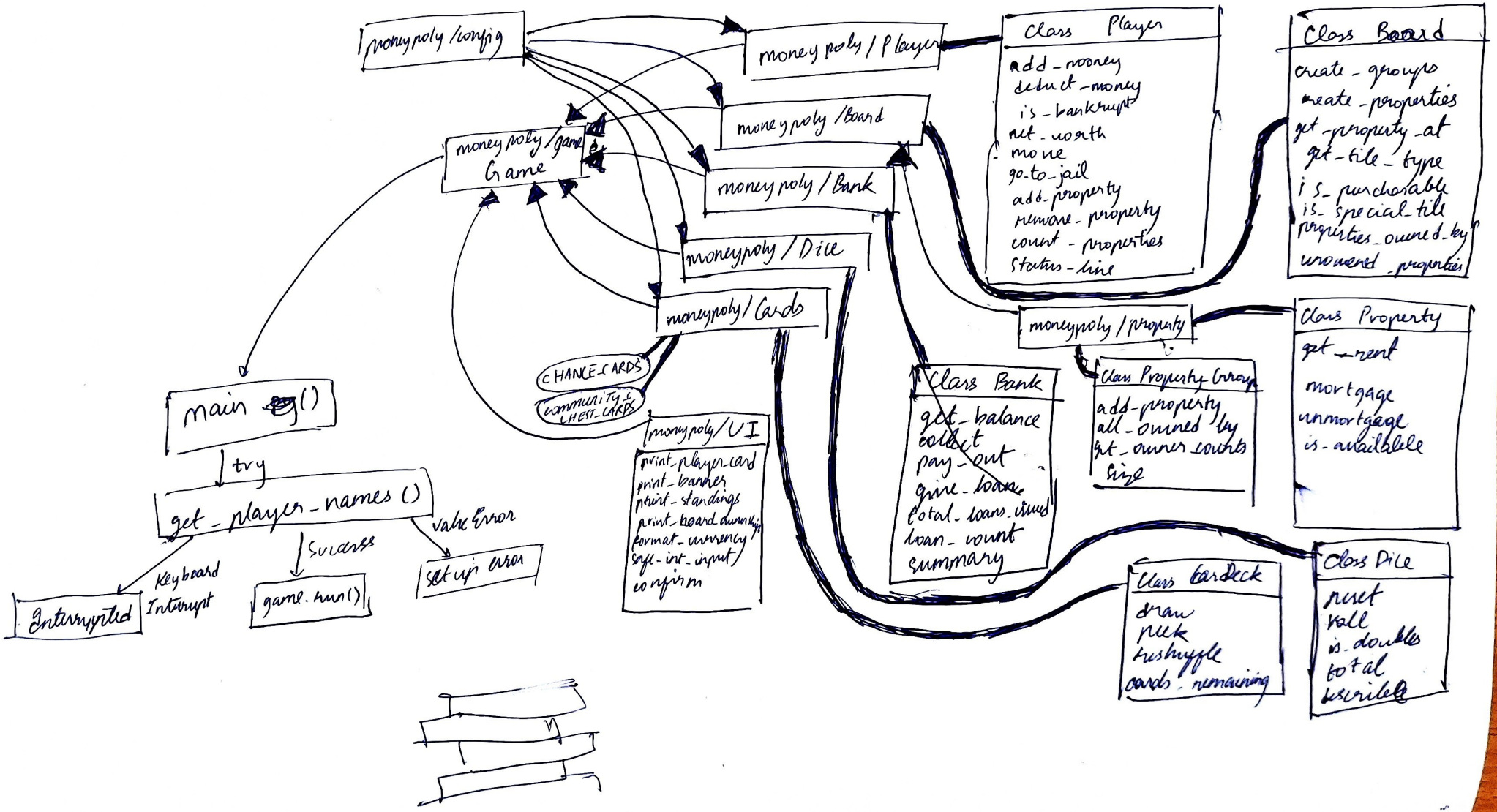
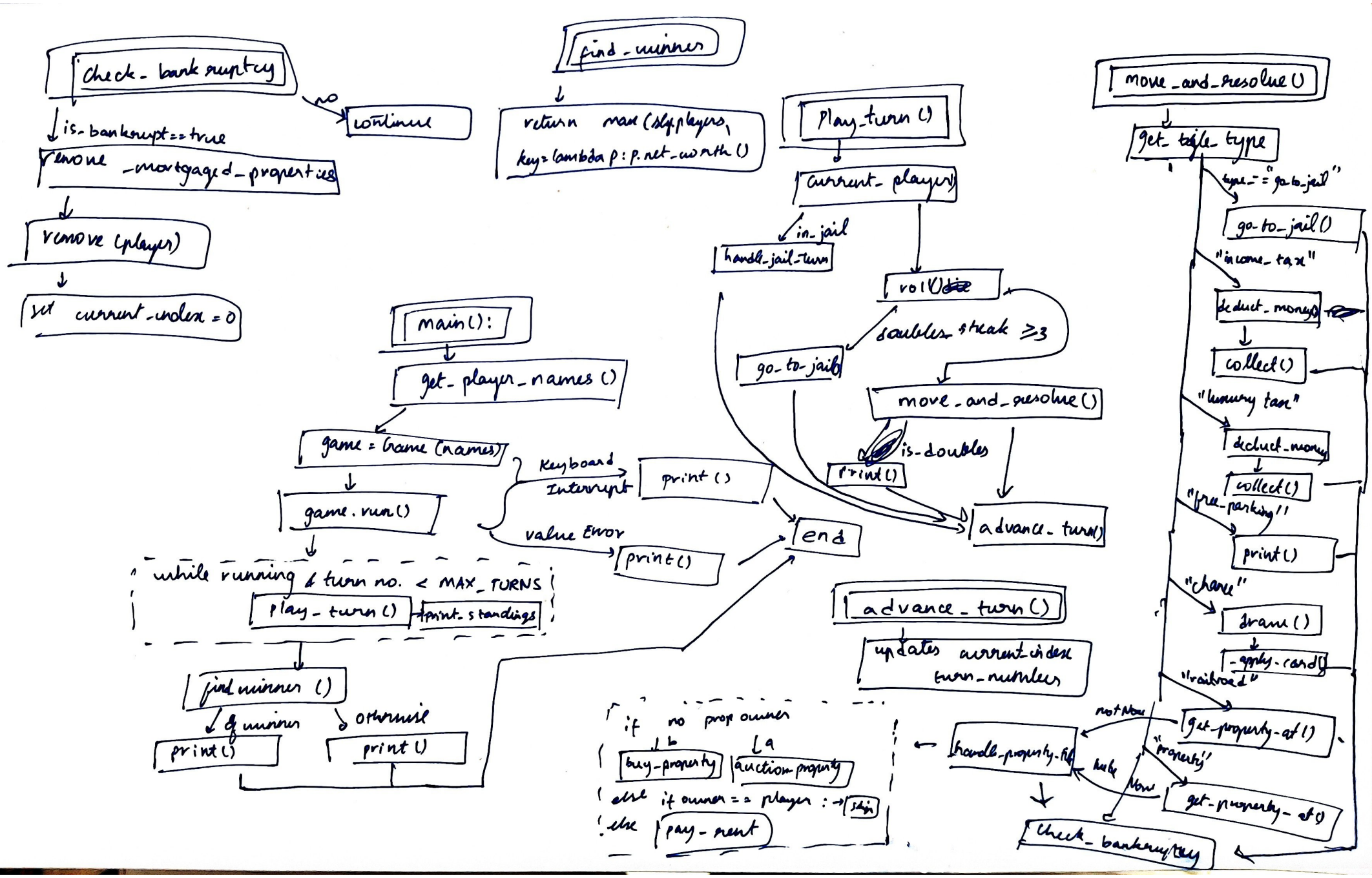

monopoly/config
monopoly/Player
Class Player
add_money
deduct_money
is_bankrupt
net_worth
move
go_to_jail
add_property
remove_property
count_properties
status_line
Class Board
create_groups
create_properties
get_property_at
get_tile_type
is_purchasable
is_special_tile
properties_owned_by
unowned_properties
monopoly/Board
monopoly/Game
Game
monopoly/Bank
monopoly/Dice
monopoly/Cards
CHANCE_CARDS
COMMUNITY_CHEST_CARDS
monopoly/Property
Class Property
get_rent
mortgage
unmortgage
is_available
Class Bank
get_balance
collect
pay_out
give_loan
total_loans_issued
loan_count
summary
Class PropertyGroup
add_property
all_owned_by
get_owner_counts
size
monopoly/UI
print_player_card
print_banner
print_standings
print_board_summary
format_currency
safe_int_input
confirm
main()
try
get_player_names()
ValueError
Success
Keyboard Interrupt
Interrupted
game.run()
set up error
Class CardDeck
draw
peek
shuffle
cards_remaining
Class Dice
reset
roll
is_doubles
total
describe

CFG:


check_bankruptcy
no
continue
is_bankrupt == true
remove_mortgaged_properties
remove(player)
set current_index = 0
find_winner
return max(alg.players, key=lambda p: p.net_worth())
Play_turn()
current_player()
in_jail
handle_jail_turn
roll()
doubles_streak ≥ 3
go_to_jail
move_and_resolve()
is_doubles
print()
advance_turn()
main():
get_player_names()
game = Game(names)
game.run()
Keyboard Interrupt
print()
value Error
print()
end
while running & turn no. < MAX_TURNS
play_turn()
print_standings
find_winner()
if winner
print()
otherwise
print()
advance_turn()
updates current_index turn_number
if no prop owner
b
buy_property
a
auction_property
else if owner == player : → skip
else
pay_rent
handle_property_fld
move_and_resolve()
get_tile_type
type == "go_to_jail"
go_to_jail()
"income_tax"
deduct_money()
collect()
"luxury tax"
deduct_money
collect()
"free_parking"
print()
"chance"
draw()
apply_card()
"railroad"
get_property_at()
"property"
get_property_at()
not Mine
Mine
Check_bankruptcy